11a. Overview and Goals

Martin Alfaro
PhD in Economics

The preceding chapter explored *single-core* parallelization through SIMD, which allows one processor core to perform the same computation simultaneously across multiple data elements. This chapter shifts focus to *multi-core* parallelization, where programs can execute simultaneously on several processor cores.

By default, most programming languages follow a sequential model: a single path of execution that advances one operation at a time. This linear approach simplifies reasoning about program behavior, as each operation completes before the next begins. However, hardware these days is commonly equipped with multiple processor cores. Consequently, a sequential execution does all the work on one core, while the others sit idle. This leaves substantial computational power untapped.

Multithreading addresses this limitation by simultaneously running different segments of a program across multiple cores. While this capability opens up significant opportunities for performance improvement, it also introduces new challenges that developers need to navigate carefully.

Compared to SIMD, **multithreading introduces even greater complexity**. Because multiple threads share the same memory space, execution can fail when operations are dependent. Thus, simple operations that work flawlessly in single-threaded programs may yield incorrect results in a multithreaded setting. In this context, writing multithreaded code requires a shift in how users think about execution. All this makes multithreaded code inherently more difficult to design, test, and debug than its single-threaded counterpart.

Despite these challenges, multithreading is essential for harnessing the full potential of modern processors. In particular, it's valuable for computationally intensive applications or when the same operation must be applied to multiple independent objects. The key is to recognize when multithreading is appropriate and how to avoid its pitfalls.

The roadmap of the chapter is as follows:

- Section 11b introduces the foundational concepts needed to reason about multithreading, including tasks and threads.
- Section 11c introduces task-based parallelism, implemented through the `@spawn` macro. This macro generates tasks that execute concurrently, with each task scheduled to run as soon as a thread becomes available. The section also emphasizes the overhead involved in managing tasks, implying that multithreading is only effective for large workloads.
- Section 11d examines which operations can be parallelized safely. Dependent operations may yield incorrect results when executed in parallel, turning common operations like reductions unsafe under naive parallelization. In contrast, "embarrassingly parallel problems", whose iterations are completely independent, can be parallelized seamlessly.

- The following sections transition to practical implementation. Section 11e compares two approaches to parallelizing for-loops: the `@threads` macro and `@spawn`. Building on this, Section 11f addresses how to parallelize for-loops effectively, focusing on embarrassingly parallel problems and reductions.
- Acknowledging the complexity of parallelizing code, Section 11g explores packages that simplify multithreading. They free developers from low-level implementation details. Notable examples are the `OhMyThreads` package and `@tturbo` macro from `LoopVectorization`. The latter combines SIMD and multithreading for maximum performance.